SOFTWARE INGENIARITZA II
2026KO URRIA

ERREFAKTORIZAZIOA

Aitzol Waliño eta Angela Noriega



AURKIBIDEA

1.WRITE SHORT UNITS OF CODE.....2
2.WRITE SIMPLE UNITS OF CODE.....3
3.DUPLICATE CODE.....3
4.KEEP UNIT INTERFACES SMALL.....4

1.WRITE SHORT UNITS OF CODE

ANGELA

Hasierako kodea:

```
	public boolean erregistratu(String izena, String abizena, Date jaiotzeData, String sexua, String erabiltzaileMota,
String email, String pasahitza) {
		open();
		if(erabiltzaileMota.equals("Gidaria")) {
			Driver u = db.find(Driver.class,email);
			if (u==null) {
				db.getTransaction().begin();
				Driver d=new Driver(izena, abizena, jaiotzeData, sexua, email, pasahitza);
				db.persist(d);
				db.getTransaction().commit();
				close();
				return true;
			} else {
				return false;
			}
		} else {
			Bidaiaria u = db.find(Bidaiaria.class,email);
			if (u==null) {
				db.getTransaction().begin();
				Bidaiaria b=new Bidaiaria(izena, abizena, jaiotzeData, sexua, email, pasahitza);
				db.persist(b);
				db.getTransaction().commit();
				close();
				return true;
			} else {
				return false;
			}
		}
	}
```

Erregistratu metodoak 23 kode-lerro zituen. Gidariaren eta bidaiariaren kudeaketentzako 15 kode lerro baino gutxiago dituzten bi metodo berri sortu ditugu, Erregistratu metodoaren kode lerroak 4ra murriztuz.

```
	public boolean erregistratu(String izena, String abizena, Date jaiotzeData, String sexua, String erabiltzaileMota,
String email, String pasahitza) {
		open();
		if(erabiltzaileMota.equals("Gidaria")) {
			return gordeGidaria(izena, abizena, jaiotzeData, sexua, email, pasahitza);
		}
		return gordeBidaiaria(izena, abizena, jaiotzeData, sexua, email, pasahitza);

	}
	private boolean gordeBidaiaria(String izena, String abizena, Date jaiotzeData, String sexua, String email,
			String pasahitza) {
		Bidaiaria u = db.find(Bidaiaria.class,email);
		if (u==null) {
			db.getTransaction().begin();
			Bidaiaria b=new Bidaiaria(izena, abizena, jaiotzeData, sexua, email, pasahitza);
			db.persist(b);
			db.getTransaction().commit();
			close();
			return true;
		} else {
			return false;
		}
	}
```

```
	private boolean gordeGidaria(String izena, String abizena, Date jaiotzeData, String sexua, String email,
			String pasahitza) {
		Driver u = db.find(Driver.class,email);
		if (u==null) {
			db.getTransaction().begin();
			Driver d=new Driver(izena, abizena, jaiotzeData, sexua, email, pasahitza);
			db.persist(d);
			db.getTransaction().commit();
			close();
			return true;
		} else {
			return false;
		}
	}
```

AITZOL

Hasierako Kodea:

```
// GEHITU
public void egoeraEzarri(int errekzbk, String egoera) throws erreklamazioaEbatzitaException {
    open();
    db.getTransaction().begin();
    Erreklamazioa e = db.find(Erreklamazioa.class, errekzbk);
    if(e.getEgoera().equals("itxaron")) {
        e.setEgoera(egoera);
        if(egoera.equals("onartu")) {
            float diruIzoztua = e.getErreserbarenDiruIzoztua();
            Bidaiaria b = e.getNork();
            Driver d = e.getNori();
            b.diruaGehitu(diruIzoztua);
            b.addMugimendua("Dirua itzuli zaizu erreklamazioaren onarpenarengatik");
            d.diruaKendu(diruIzoztua);
            d.addMugimendua("Dirua kendu zaizu erreklamazioaren onarpenarengatik");
        }
        else if(egoera.equals("deuseztatu")) {
            /*TypedQuery<Admin> aquery = db.createQuery("SELECT a FROM Admin a", Admin.class);
            Admin a = aquery.getResultList().get(0);
            a.addErreklamazioa(e);*/
            System.out.println("Egoera deuseztatu da.");
        }
    } else {
        throw new erreklamazioaEbatzitaException();
    }
    db.getTransaction().commit();
    close();
}
```

egoeraEzarri metodoak 19 kode-lerro zituen. Bi egoeren kudeaketarako (onartu eta deuseztatu) eta diruaren itzulera gestionatzeko beste bi metodo berri sortu dira, non, biak ala biak 15 kode-lerro baino gutxiago dituzten. Gainera, egoeraEzarri metodoa 9 kode-lerro izatera pasa da.

```
public void egoeraEzarri(int errekzbk, String egoera) throws erreklamazioaEbatzitaException {
    open();
    db.getTransaction().begin();
    Erreklamazioa e = db.find(Erreklamazioa.class, errekzbk);
    if (!e.getEgoera().equals("itxaron")) {
        throw new erreklamazioaEbatzitaException();
    }
    e.setEgoera(egoera);
    egoeraBerriaKudeatu(e, egoera);
    db.getTransaction().commit();
    close();
}
```

```
private void egoeraBerriaKudeatu(Erreklamazioa e, String egoera) {
    if (egoera.equals("onartu")) {
        diruaItzuli(e);
    } else if (egoera.equals("deuseztatu")) {
        System.out.println("Egoera deuseztatu da.");
    }
}

private void diruaItzuli(Erreklamazioa e) {
    float diruIzoztua = e.getErreserbarenDiruIzoztua();
    Bidaiaria b = e.getNork();
    Driver d = e.getNori();
    b.diruaGehitu(diruIzoztua);
    b.addMugimendua("Dirua itzuli zaizu erreklamazioaren onarpenarengatik");
    d.diruaKendu(diruIzoztua);
    d.addMugimendua("Dirua kendu zaizu erreklamazioaren onarpenarengatik");
}
```

2.WRITE SIMPLE UNITS OF CODE

Ez daukagu 4ko konplexutasun ziklomatikoa gainditzen duen koderik. Hortaz atal hau ezin dugu egin.

3.DUPLICATE CODE

ANGELA

Hasierako kodea:

```
    private boolean gordeBidaiaria(String izena, String abizena, Date jaiotzeData, String sexua, String email,
                    String pasahitza) {
            Bidaiaria u = db.find(Bidaiaria.class,email);
            if (u==null) {
                    db.getTransaction().begin();
                    Bidaiaria b=new Bidaiaria(izena, abizena, jaiotzeData, sexua, email, pasahitza);
                    db.persist(b);
                    db.getTransaction().commit();
                    close();
                    return true;
            } else {
                    return false;
            }
    }
    private boolean gordeGidaria(String izena, String abizena, Date jaiotzeData, String sexua, String email,
                    String pasahitza) {
            Driver u = db.find(Driver.class,email);
            if (u==null) {
                    db.getTransaction().begin();
                    Driver d=new Driver(izena, abizena, jaiotzeData, sexua, email, pasahitza);
                    db.persist(d);
                    db.getTransaction().commit();
                    close();
                    return true;
            } else {
                    return false;
            }
```

```
	}
```

Objektuak datu basean sartzeko bi aldiz berdina egiten dugu. Metodo berri bat sortu dugu, hori maneiatzeko eta leku bakarrean idatzita egoteko.

```
	private boolean gordeBidaiaria(String izena, String abizena, Date jaiotzeData, String sexua, String email,
				String pasahitza) {
		Bidaiaria u = db.find(Bidaiaria.class,email);
		if (u==null) {
				Bidaiaria b=new Bidaiaria(izena, abizena, jaiotzeData, sexua, email, pasahitza);
				persistitu(b);
				close();
				return true;
		} else {
				return false;
		}
	}
	private boolean gordeGidaria(String izena, String abizena, Date jaiotzeData, String sexua, String email,
				String pasahitza) {
		Driver u = db.find(Driver.class,email);
		if (u==null) {
				Driver d=new Driver(izena, abizena, jaiotzeData, sexua, email, pasahitza);
				persistitu(d);
				close();
				return true;
		} else {
				return false;
		}
	}

	private void persistitu(Object o) {
		db.getTransaction().begin();
		db.persist(o);
		db.getTransaction().commit();
	}
```

AITZOL

Hasierako Kodea:

```
public ObjectdbManagerServer() {

    setTitle("objectDBManagerServer: running the database server");
    setBounds(100, 100, 486, 180);
    getContentPane().setLayout(new BorderLayout());
    contentPanel.setBorder(new EmptyBorder(5, 5, 5, 5));
    getContentPane().add(contentPanel, BorderLayout.CENTER);
    contentPanel.setLayout(new BorderLayout(0, 0));
    {
        textArea = new JTextArea();
        contentPanel.add(textArea);
    }
    {
        JPanel buttonPane = new JPanel();
        buttonPane.setLayout(new FlowLayout(FlowLayout.RIGHT));
        getContentPane().add(buttonPane, BorderLayout.SOUTH);
        {
            JButton okButton = new JButton("OK");
            okButton.addActionListener(new ActionListener() {
                public void actionPerformed(ActionEvent e) {
                    textArea.append("\n\n\nClosing the database... ");
                    try {
                        System.out.println("Server close");
                         try {


                                Runtime.getRuntime().exec("java -cp "+objectDbpath+" com.objectdb.Server -port "+ c.getDatabasePort()+" stop");

                            } catch (Exception ioe) {
                                System.out.println (ioe);
                            }
                            System.exit(1);

                    } catch (Exception e1) {
                    }
                    System.exit(1);
                }
            });
            okButton.setActionCommand("OK");
            buttonPane.add(okButton);
            getRootPane().setDefaultButton(okButton);
        }
        {
            JButton cancelButton = new JButton("Cancel");
            cancelButton.setActionCommand("Cancel");
            buttonPane.add(cancelButton);
        }
    }

    ConfigXML c=ConfigXML.getInstance();

    if (c.isDatabaseLocal()) {
        textArea.append("\nERROR, the database is configured as local");
    }
    else {
    try{
        System.out.println("Lauching objectdb server");

        try {
            Runtime.getRuntime().exec("java -cp "+objectDbpath+" com.objectdb.Server -port "+ c.getDatabasePort()+" start");
        } catch (Exception ioe) {
            System.out.println (ioe);
```

objectdbManagerServer klasean kodearen zati bat errepikakorra da eta horretarako metodo bat sortu dugu hori ekiditeko eta toki bakarrean egoteko idatzita.

```
public ObjectdbManagerServer() {

    setTitle("objectDBManagerServer: running the database server");
    setBounds(100, 100, 486, 180);
    getContentPane().setLayout(new BorderLayout());
    contentPanel.setBorder(new EmptyBorder(5, 5, 5, 5));
    getContentPane().add(contentPanel, BorderLayout.CENTER);
    contentPanel.setLayout(new BorderLayout(0, 0));
    {
        textArea = new JTextArea();
        contentPanel.add(textArea);
    }
    {
        JPanel buttonPane = new JPanel();
        buttonPane.setLayout(new FlowLayout(FlowLayout.RIGHT));
        getContentPane().add(buttonPane, BorderLayout.SOUTH);
        {
            JButton okButton = new JButton("OK");
            okButton.addActionListener(new ActionListener() {
                public void actionPerformed(ActionEvent e) {
                    textArea.append("\n\n\nClosing the database... ");
                    try {
                        System.out.println("Server close");
                        kudeatu("stop");
                            System.exit(1);

                    } catch (Exception e1) {
                    }
                    System.exit(1);
                }
            });
            okButton.setActionCommand("OK");
            buttonPane.add(okButton);
            getRootPane().setDefaultButton(okButton);
        }
        {
            JButton cancelButton = new JButton("Cancel");
            cancelButton.setActionCommand("Cancel");
            buttonPane.add(cancelButton);
        }
    }

    ConfigXML c=ConfigXML.getInstance();

    if (c.isDatabaseLocal()) {
        textArea.append("\nERROR, the database is configured as local");
    }
    else {
    try{
        System.out.println("Lauching objectdb server");

        kudeatu("start");

        textArea.append("\nAccess granted to: "+c.getUser());

        textArea.append("\nPress button to exit this database server... ");

    } catch (Exception e) {
        textArea.append("Something has happened in ObjectDbManagerServer: "+e.toString());
    }
```

4.KEEP UNIT INTERFACES SMALL

ANGELA

Hasierako kodea:

```
        public boolean erregistratu(String izena, String abizena, Date jaiotzeData, String sexua, String erabiltzaileMota,
String email, String pasahitza) {
                open();
                if(erabiltzaileMota.equals("Gidaria")) {
                        return gordeGidaria(izena, abizena, jaiotzeData, sexua, email, pasahitza);
                }
                return gordeBidaiaria(izena, abizena, jaiotzeData, sexua, email, pasahitza);

        }
        private boolean gordeBidaiaria(String izena, String abizena, Date jaiotzeData, String sexua, String email,
                        String pasahitza) {
```

```
                Bidaiaria u = db.find(Bidaiaria.class,email);
                if (u==null) {
                        db.getTransaction().begin();
                        Bidaiaria b=new Bidaiaria(izena, abizena, jaiotzeData, sexua, email, pasahitza);
                        db.persist(b);
                        db.getTransaction().commit();
                        close();
                        return true;
                } else {
                        return false;
                }
        }
        private boolean gordeGidaria(String izena, String abizena, Date jaiotzeData, String sexua, String email,
                        String pasahitza) {
                Driver u = db.find(Driver.class,email);
                if (u==null) {
                        db.getTransaction().begin();
                        Driver d=new Driver(izena, abizena, jaiotzeData, sexua, email, pasahitza);
                        db.persist(d);
                        db.getTransaction().commit();
                        close();
                        return true;
                } else {
                        return false;
                }
        }
```

Erregistratu metodoak 7 parametro jasotzen ditu, errefraktorizazio honek 4 baino gutxiago izatea eskatzen digu. Hau zuzentzeko parametroak gordetzen dituen klase bat sortu dugu. Lehen sortutako bi metodo berriak baita zuzendu ditugu.

```
public boolean erregistratu(ErregistratuParameter parameterObject) {
                open();
                if(parameterObject.erabiltzaileMota.equals("Gidaria")) {
                        return gordeGidaria(parameterObject);
                }
                return gordeBidaiaria(parameterObject);

        }
        private boolean gordeBidaiaria(ErregistratuParameter parameterObject) {
                Bidaiaria u = db.find(Bidaiaria.class,parameterObject.email);
                if (u==null) {
                        Bidaiaria b=new Bidaiaria(parameterObject.izena,
parameterObject.abizena,parameterObject.jaiotzeData, parameterObject.sexua, parameterObject.email,
parameterObject.pasahitza);
                        persistitu(b);
                        close();
                        return true;
                } else {
                        return false;
                }
        }
        private boolean gordeGidaria(ErregistratuParameter parameterObject) {
                Driver u = db.find(Driver.class,parameterObject.email);
                if (u==null) {
                        Driver d=new Driver(parameterObject.izena, parameterObject.abizena,
parameterObject.jaiotzeData, parameterObject.sexua, parameterObject.email, parameterObject.pasahitza);
                        persistitu(d);
                        close();
                        return true;
                } else {
                        return false;
```

```
                }
        }
```

```
package dataaccess;
import java.util.Date;
public class ErregistratuParameter {
        public String izena;
        public String abizena;
        public Date jaiotzeData;
        public String sexua;
        public String erabiltzaileMota;
        public String email;
        public String pasahitza;
        public ErregistratuParameter(String izena, String abizena, Date jaiotzeData, String sexua, String erabiltzaileMota,
                        String email, String pasahitza) {
                this.izena = izena;
                this.abizena = abizena;
                this.jaiotzeData = jaiotzeData;
                this.sexua = sexua;
                this.erabiltzaileMota = erabiltzaileMota;
                this.email = email;
                this.pasahitza = pasahitza;
        }
}
```

AITZOL

Hasierako kodea:



```
public Ride createRide(String from, String to, Date date, int nPlaces, float price, String driverEmail) throws  RideAlreadyExistException, RideMustBeLaterThanTodayException {
    System.out.println(">> DataAccess: createRide=> from= "+from+" to= "+to+" driver="+driverEmail+" date "+date);
    try {
        if(new Date().compareTo(date)>0) {
            throw new RideMustBeLaterThanTodayException(ResourceBundle.getBundle("Etiquetas").getString("CreateRideGUI.ErrorRideMustBeLaterThanToday"));
        }
        db.getTransaction().begin();

        Driver driver = db.find(Driver.class, driverEmail);
        if (driver.doesRideExists(from, to, date)) {
            db.getTransaction().commit();
            throw new RideAlreadyExistException(ResourceBundle.getBundle("Etiquetas").getString("DataAccess.RideAlreadyExist"));
        }
        Ride ride = driver.addRide(from, to, date, nPlaces, price);
        //next instruction can be obviated
        db.persist(driver);
        db.getTransaction().commit();

        return ride;
    } catch (NullPointerException e) {
        // TODO Auto-generated catch block
        db.getTransaction().commit();
        return null;
    }


}
```

createRide metodoak 6 parametro jasotzen ditu eta errefaktorizazio honek 4 parametro baino gutxiago eskatzen ditu Hau konpontzeko beste klase bat sortu dugu parametro hauek jasotzen dituena.

```java
public Ride createRide(createRideParameter obj, String driverEmail) throws  RideAlreadyExistException, RideMustBeLaterThanTodayException {
    System.out.println(">> DataAccess: createRide=> from= "+obj.getFrom()+" to= "+obj.getTo()+" driver="+driverEmail+" date "+obj.getDate());
    try {
        if(new Date().compareTo(obj.getDate())>0) {
            throw new RideMustBeLaterThanTodayException(ResourceBundle.getBundle("Etiquetas").getString("CreateRideGUI.ErrorRideMustBeLaterThanToday"));
        }
        db.getTransaction().begin();

        Driver driver = db.find(Driver.class, driverEmail);
        if (driver.doesRideExists(obj.getFrom(), obj.getTo(), obj.getDate())) {
            db.getTransaction().commit();
            throw new RideAlreadyExistException(ResourceBundle.getBundle("Etiquetas").getString("DataAccess.RideAlreadyExist"));
        }
        Ride ride = driver.addRide(obj.getFrom(), obj.getTo(), obj.getDate(), obj.getNPlaces(), obj.getPrice());
        //next instruction can be obviated
        db.persist(driver);
        db.getTransaction().commit();

        return ride;
    } catch (NullPointerException e) {
        // TODO Auto-generated catch block
        db.getTransaction().commit();
        return null;
    }


}
```

```java
package dataaccess;

import java.util.Date;

public class createRideParameter {
        private String from;
        private String to;
        private Date date;
        private int nPlaces;
        private float price;

        public createRideParameter(String from, String to, Date date, int nPlaces, float price) {
            this.from = from;
            this.to = to;
            this.date = date;
            this.nPlaces = nPlaces;
            this.price = price;
        }

        public String getFrom()  {
            return from;
        }
        public String getTo() {
            return to;
        }
        public Date getDate() {
            return date;
        }
        public int getNPlaces() {
            return nPlaces;
        }
        public float getPrice() {
            return price;
        }
}
```